

SEMWEAVER: Provenance-Guided Refinement of LLM-Generated Vulnerability Checkers

ANONYMOUS AUTHOR(S)

LLM-generated static vulnerability checkers can warn on both vulnerable and fixed code, or remain silent on vulnerable code. We present SEMWEAVER, an automated post-synthesis refinement workflow, and characterize the benefits and costs of exposing analyzer context to its editor. The workflow combines patch-local, paired validation with provenance-labelled patch context, analyzer outputs, and analyzer-internal evidence: backend-produced representations and checker-execution observations bound to raw output and source scope. It retains warning-recovering or fixed-noise-reducing edits only after normal execution on both revisions.

We evaluate all 39 frozen, generated-only KNightier CSA checkers: 15 initially warn on both revisions and 24 are silent on both. With the same GPT-6-Luna model, SEMWEAVER reaches 23 warning-positive vulnerable revisions versus 15 for KNightier’s actual refinement loop, with patch-version $F_1 = 0.575$ versus 0.448. In the preexisting 15-case fixed-noisy stratum, SEMWEAVER retains vulnerable-side warning signals with 44 fixed-side reports versus KNightier’s 69; the reduction is concentrated in one subject. Native versus no-internal refinement yields six comparative benefits, five disadvantages and nearly identical aggregate F_1 ; native’s additional coverage also increases overall fixed-side noise. Three paired decodes on 12 subjects show sampling-dependent effects. A separate automatic CodeQL example and three-model subset examine feasibility and configuration sensitivity. Together, the comparisons characterize conditional checker-completion gains and the tradeoff between warning recovery and inspection burden. Independently adjudicated target recall, universally superior precision and unseen-site generalization remain unestablished.

CCS Concepts: • **Software and its engineering** → **Automated static analysis**; • **Security and privacy** → **Systems security**.

Additional Key Words and Phrases: static analysis, vulnerability checkers, large language models, patch-guided refinement

1 Introduction

Static analysis is a cornerstone of software security assurance for systems code. Tools such as CodeQL [18] and Clang Static Analyzer (CSA) [37] detect vulnerabilities by checking source code against predefined rules without executing it [4, 7, 15, 48]. The effectiveness of these tools depends on the quality of their vulnerability checkers, each of which must encode the semantic conditions (path guards, state invariants, data-flow relations, and API contracts) that distinguish a real bug from safe code. Writing such checkers by hand requires deep expertise in both the target codebase and the analysis framework, making manual authoring expensive, slow, and difficult to scale.

LLM-based patch-driven synthesis offers a promising alternative. In this paradigm, a security patch serves as a compact before/after specification: the LLM consumes the patch and synthesizes a reusable checker that should fire on the vulnerable revision but remain silent on the fixed revision. KNightier [57], a representative system, has demonstrated that this approach can produce compilable and executable CSA checkers from historical Linux kernel fixes.

However, the resulting checkers may be semantically inadequate. A security patch is inherently underspecified: a one-line goto-target change may encode an ownership barrier, a widened cast may encode a range guard, and a moved free may encode a lifetime invariant. A checker that captures only the syntactic shape of the edit, without encoding the underlying semantic condition, can fire on both the vulnerable and fixed revisions. After naïve refinement, the checker may instead

lose the vulnerable-side trigger entirely, a failure mode we call over-pruning. These outcomes can arise from incomplete API models, incorrect state updates, or a missing relation between the patch and the detector’s trigger.

Iterative LLM refinement can use self-generated or external feedback [40, 52]. For checker refinement, KNighter uses execution reports and compiler feedback to guide edits [57]; its report-driven loop does not attempt an edit when no report is available. The observed warning identifies a failed behavior, but may leave several plausible causes: a missing path guard, cleanup barrier, ownership transition, or API condition. An editor can reason from its patch and source context; additional analyzer representations provide another inspectable source of relevant structure.

We make a narrower observation: **some patch-relevant program structure is available in analyzer-produced intermediate representations** but is not ordinarily surfaced to the refinement loop. Path-sensitive analyzers such as CSA internally track branch guards, symbolic states, memory regions, and lifetime events during checker execution [37]. Relational query engines such as CodeQL maintain databases of data-flow edges, call-graph relations, type hierarchies, and API-usage patterns [2, 18]. These representations can suggest candidate predicates, but a CFG edge or call relation is not by itself a proof of path feasibility or checker correctness.

Exploiting these intermediate results, however, is not straightforward. Analyzer dumps and database relations can be much larger than the relevant patch scope. Selecting a bounded view also requires preserving its source binding and distinguishing backend-produced structure from diagnostic or source-derived context. The challenge is to select and present only the evidence that is relevant to the particular semantic gap in the checker under refinement.

SEMWEAVER provides post-synthesis checker editing guided by provenance-labelled analyzer context. SEMWEAVER is designed as a plug-in post-generation layer that is independent of the upstream LLM generator, treating any generated checker (from KNighter or another pipeline) as an artifact to be refined rather than regenerated. The contribution couples this refinement workflow with an empirical account of its benefits and costs. The workflow addresses evidence selection through three mechanisms. First, a rule-guided preflight extracts patch facts (e.g., guards, type widening, and affected calls) and prioritizes evidence requirements by inferred mechanism. The editor may request role views in the general workflow; the strict CSA comparison instead freezes its patch-scoped view before editing. For instance, a cleanup-edge patch triggers collection of path-guard and lifetime evidence from CSA, while a bounds patch triggers guard, flow, and API-usage evidence from CodeQL. Second, backend outputs, patch facts, source context, and validation feedback are normalized into a compact bundle with typed records, giving the editor a structured edit target (“add the patch-introduced cleanup barrier while preserving the freed-object transition”) instead of a vague behavioral symptom (“fixed side still warns”). Third, paired validation retains a candidate when it recovers a previously absent vulnerable-side warning, or reduces fixed-side reports while retaining vulnerable-side warning presence. Fixed-side silence is a stronger secondary outcome, not a prerequisite for every useful edit. This operational constraint does not independently establish target-vulnerability detection.

We evaluate all 39 frozen, generated-only KNighter CSA checkers. Fifteen initially warn on both revisions and 24 are silent on both; the initial zero PDS is a starting condition, not KNighter’s refinement result. All methods share starting artifacts and requested GPT-6-Luna high, with a 32-reply ceiling and their recorded stopping policies. Native SEMWEAVER reaches 23 warning-positive vulnerable revisions and version-level $F_1 = 0.575$, versus 15 and 0.448 for KNighter’s actual loop. The complete 15-case report-refinement stratum retains all vulnerable-side warning signals with 44 fixed-side reports versus KNighter’s 69, mainly through G24. Across all 39, native’s 123 fixed-side reports exceed KNighter’s 69 and no-internal’s 68. These contrasts expose distinct outcomes: a

workflow can recover vulnerable-side warnings while increasing inspection burden, and reduce report counts without changing binary discrimination. The native/no-internal comparison and three paired decodes on the original 12 subjects document conditional benefits and sampling sensitivity; they establish no universal native superiority. Separate automatic CodeQL and fixed-subset model runs examine feasibility and configuration sensitivity.

This paper makes the following contributions:

- We distinguish output-level behavioral feedback from provenance-bound analyzer-internal evidence, and formulate paired-warning refinement for vulnerability checkers generated by LLMs.
- We present SEMWEAVER, a plug-in refinement layer that organizes provenance-labelled analyzer representations, analyzer outputs, patch and source context, and validation feedback into a compact semantic evidence bundle; the strict CSA comparison freezes the available view rather than evaluating adaptive routing.
- We characterize the benefits and costs of evidence-guided refinement through an automatic, denominator-complete 39-checker comparison with KNighter’s actual loop, a full-cohort ablation and repeated-subset analysis. Recorded code changes connect warning recovery and report reduction to concrete checker repairs and broader matching; mixed outcomes and cross-backend limits remain visible.

2 Background

2.1 Static Vulnerability Checkers

Static analysis examines source code without executing it, aiming to find bugs, security vulnerabilities, and code-quality issues at compile time or during code review [4, 15]. A vulnerability checker is a rule or query within a static-analysis framework that flags program locations matching a security-relevant condition. Modern frameworks support different checker paradigms. Path-sensitive engines such as the Clang Static Analyzer (CSA) perform symbolic exploration over program paths, tracking branch guards, symbolic values, memory regions, and lifetime transitions to detect bugs like use-after-free, null dereference, and resource leaks [37]. Relational query engines such as CodeQL compile source code into a queryable database and express checkers as declarative queries over data-flow edges, call-graph relations, type hierarchies, and API-usage patterns [2, 18]. Static-analysis frameworks have been deployed at industrial scale for continuous bug detection [7, 48].

Writing effective checkers remains a bottleneck. A checker must encode not only the syntactic pattern of a vulnerability but also the semantic conditions (path guards, state invariants, data-flow relations, and API contracts) that distinguish real bugs from safe code. This requires deep domain knowledge of both the target codebase and the analysis framework, making manual checker authoring expensive and error-prone. Report burden also affects practical use [20, 23]; ranking and actionability prediction address emitted warnings [29, 47], whereas our refinement changes checker code.

2.2 LLM-Based Patch-Driven Checker Generation

Recent work has proposed using LLMs to automate checker generation from security patches. A security patch provides a compact before/after specification: the vulnerable revision should trigger the generated checker, and the fixed revision should not. This idea builds on a well-established line of work that treats patches as vulnerability specifications, including semantic patching [43], clone-based vulnerability discovery [22, 28], and patch-derived datasets [16]. KNighter [57] is a

representative system that synthesizes CSA checkers from historical Linux kernel fixes and validates them against the vulnerable and fixed revisions.

Generated checkers may miss semantic conditions: a small patch can encode a guard, ownership transition, lifetime barrier or API contract beyond its syntactic shape. KNighter’s refinement loop operates at the output level [57]. It feeds behavioral signals (e.g., “the fixed side still warns”) back to the LLM, revealing the symptom but not diagnosing the cause. Such feedback does not directly identify whether the missing predicate concerns a path guard, a cleanup barrier, an ownership transition, or an API contract; the model must infer the condition from the patch, source context and observed outputs.

2.3 Analyzer-Internal Intermediate Results

Static-analysis backends compute intermediate representations that can be exposed through debugging interfaces or database queries, but these are not normally supplied to a checker-refinement loop.

CSA, as a path-sensitive engine, internally maintains per-path symbolic states that record branch conditions, memory region bindings, allocation/free events, and object lifetime transitions [37]. CodeQL, as a relational engine, materializes a full program database from which data-flow edges, call-graph relations, type hierarchies, member-access patterns, and API-usage sequences can be queried [2, 18]. These intermediate facts have long underpinned taint analysis, graph-based detection, and interprocedural security reasoning [1, 25, 36, 55].

These representations may suggest a missing predicate, but a CFG branch or database tuple does not itself establish vulnerability semantics. The operational distinction between a backend-produced representation, a diagnostic, and a source-window fallback is central to our evidence accounting (Section 4.2). This motivates extracting bounded, provenance-labelled context for refinement while retaining normal paired execution and vulnerable-side warning presence as operational acceptance checks, not proofs of target-location correctness.

3 Motivating Example

This section traces an automatic refinement of the KNighter-derived CSA checker for Linux commit 97cba232549b, which repairs an off-by-one access in the AMD display driver [51, 57]. The example has an explicit safety relation: the largest look-ahead index must be smaller than the extent of the array actually accessed. Both the starting and refined checkers use `checkASTCodeBody`; the edit adds a missing bound relation to the existing analysis.

The bug and its patch. The member `dc->links` has declared capacity `MAX_PIPES * 2` (Figure 1, ❶). We abbreviate this constant as N in the figure; this is a presentation alias, not a new program variable. The function `get_host_router_total_dp_tunnel_bw` iterates over candidate links and, on a matching host-router branch, reads both `links[i]` and `links[i + 1]` (M10–M11, ❸). With the original condition $i < N$ (M5, ❷), the loop permits $i = N - 1$, so the look-ahead can reach index N , outside the array. The patch changes the bound to $i < N - 1$ (M6, ❹), which limits the largest look-ahead index to $N - 1$. Other branch conditions determine whether an access occurs, but the patched loop bound already prevents this final iteration from reaching an out-of-range look-ahead.

Why the baseline checker fails. The generated checker already recognizes an induction variable, a loop bound, and a look-ahead array subscript. It reports an access when it does not recognize a protective condition in an enclosing `if` (B1–B5, ❺). That rule overlooks a guard carried by the loop condition itself. The fixed program still contains `links[i + 1]`, so the checker reports on both revisions despite the reduced loop bound. The independent patch-related-object replay yields 1/1 alerts. The refinement problem is therefore to interpret the relation between the loop bound and array extent while retaining the unsafe access.

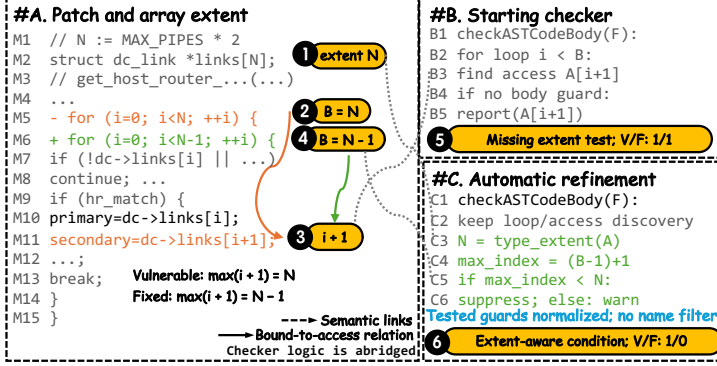


Fig. 1. **Capacity-aware look-ahead refinement.** Abridged code: N denotes $\text{MAX_PIPES} * 2$; hr_match denotes host-router equality. Original/refined vulnerable/fixed alerts are 1/1 and 1/0 in independent replay. The 14-response, model-produced illustration used fixture feedback and is separate from the one-response main-study checker. Orange/green solid arrows relate vulnerable/fixed bounds to the access; gray dotted links denote semantic correspondences. Code colors mark unsafe/patched bound and capacity logic.

Evidence-guided refinement. The available bundle contains four provenance-labelled records: two strictly internal Clang 18 CFG/call-graph records and two analyzer-output records. The internal records anchor the affected loop and branch context to raw dumps from `debug.DumpCFG` and `debug.DumpCallGraph`; patch and source context identify the bound change. The declared array extent is read from the AST type by the emitted checker. It is not a path-feasibility proof supplied by the evidence bundle.

The initial GLM-5.3-Flash edit adds a capacity-aware condition while retaining the original traversal. For the depicted loop $i < B$, the maximum look-ahead index is $(B-1)+1 = B$. A known extent N justifies suppressing the warning only when this index is smaller than N (C3–C6, ⑥). Consequently, the vulnerable $B = N$ remains reported and the fixed $B = N-1$ becomes silent. This condition depends on type and index relations rather than the patched function name, a particular array capacity, or adjacency of the relevant statements. The initial edit uses two model calls and passes the independent Linux replay at 1/0.

Checking the example beyond its patch. Additional perturbation probes expose shortcomings in the initial candidate’s fallback and inherited guard recognition. Model-driven quality continuations receive these hash-bound diagnostics and repair the checker; the raw model edits and failed intermediate versions are retained. Four automatic GPT-6-Luna-high quality continuations use 12 additional model calls (14 including the initial Flash refinement). The final frozen checker retains 1/0 on the original Linux pair. It passes all 28 core fixtures, six boundary fixtures, 20 additional guard/bound fixtures, and 60 renamed/capacity-varied executions of those 20 forms: 58/58 unsafe and 56/56 safe executions overall. The labels for the latter 80 executions were also checked with AddressSanitizer [50]. These are finite experimental fixtures, not 114 independent bugs or an unseen-data confirmation set. The fixtures and diagnostic feedback were prepared through the research workflow with coding-assistant support; every scored checker-code edit is model-produced. The tests support robustness to the tested name changes, unrelated statements, constant capacities, and equivalent unit-step-loop/guard forms. Arbitrary aliasing, dynamic extents, and non-unit loop updates remain outside this claim. This illustrative continuation is separate from the frozen 39-case

results and does not establish a causal benefit from internal records alone. The one-response main-cohort G21 checker also reports 1/0, but passes only 74/114 of these same fixtures. The stronger 14-response illustration is not substituted into the 39-case scores.

4 Approach

SEMWEAVER is a post-synthesis refinement layer for LLM-generated vulnerability detectors. Given a draft CSA checker or CodeQL query produced by an upstream generator, the patch that motivated the synthesis, and the corresponding source tree and validation scope, SEMWEAVER refines the detector to encode missing semantic predicates while retaining a vulnerable-side warning signal.

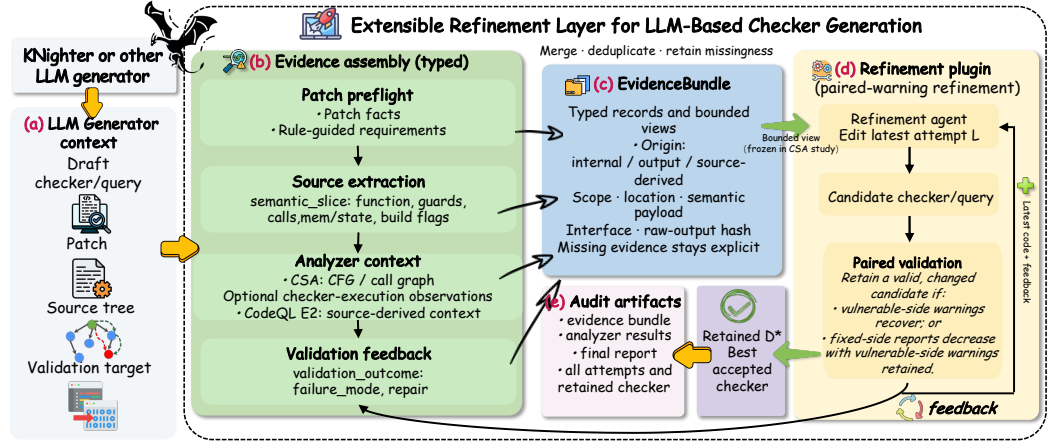


Fig. 2. **SEMWEAVER architecture.** Green groups evidence assembly, blue the provenance-labelled bundle, yellow the editor and paired validation, and purple the best retained checker; colors indicate roles, not correctness. The lower path returns latest code and feedback to the editor; accepted code also enters the audit artifacts. Retention tracks vulnerable-side warning presence and fixed-side burden, not target correctness; fixed silence is secondary. All attempts remain auditable.

Figure 2 groups the workflow into five labelled components. **(a) Frozen inputs.** The input is an existing detector, patch, source tree and validation scope, not a new generation task. **(b) Evidence assembly.** Rule-guided preflight, source extraction, analyzer collection and validation feedback supply provenance-typed records (Sections 4.1–4.2). **(c) Evidence bundle.** Normalization retains the record origin, source scope and bounded views (Section 4.3). **(d) Paired-warning refinement.** The agent edits the latest attempted detector; normal paired outcomes update a separate best detector when they recover warning presence or reduce fixed reports while retaining it. Rejected attempts remain next-step input/feedback (Section 4.4). **(e) Audit artifacts.** The verifier checks raw-output/source-scope bindings, interfaces, provenance counts, selected code and paired logs. Original prompts, replies and all attempts persist; an LLM explanation is not an origin certificate.

4.1 Evidence Selection and Exposure

The central design choice in evidence assembly (Figure 2b) is *what to collect and expose*. Analyzer outputs can be large and heterogeneous; passing them wholesale to the editor obscures patch-relevant relations. The preflight planner represents candidate requirements by role and priority,

Algorithm 1 Paired-Warning Evidence-Guided Refinement**Require:** detector D , patch P , source S , scope V , backends A , budget K **Ensure:** refined detector D^* and audit report

```

1:  $R_0 \leftarrow \text{VALIDATE}(D, P, V)$ 
2:  $E_p, E_s \leftarrow \text{PREFLIGHT}(P), \text{EXTRACTSOURCE}(P, S, D)$ 
3:  $E_a \leftarrow \bigcup_{a \in A} \text{COLLECTNATIVE}(a, D, P, S)$   $E_v \leftarrow \text{FEEDBACK}(R_0)$ 
4:  $B \leftarrow \text{NORMALIZEMERGE}(E_p, E_s, E_a, E_v)$ 
5: if  $\text{PDS}(D, P, V)$  then return  $\text{AUDIT}(D, B)$ 
6:  $D^* \leftarrow D$   $L \leftarrow D$   $T \leftarrow \text{INITIALSTOPSTATE}$ 
7: for  $i = 1$  to  $K$  do
8:    $V_i \leftarrow \text{BOUNDEDVIEW}(B)$   $C_i \leftarrow \text{AGENTEDIT}(L, P, V_i)$ 
9:    $R_i \leftarrow \text{VALIDATEIFCOMPILABLE}(C_i, P, V)$ 
10:  if  $\text{ACCEPT}(C_i, D^*)$  then  $D^* \leftarrow C_i$ 
11:   $L \leftarrow C_i$   $E_i \leftarrow \text{FEEDBACK}(R_i)$ 
12:   $B \leftarrow \text{MERGE}(B, E_i)$ 
13:   $T \leftarrow \text{UPDATESTOPSTATE}(T, R_i, D^*)$ 
14:  if  $\text{STOP}(T, K)$  then break
15: end for
16: return  $\text{AUDIT}(D^*, B)$ 

```

▷ Equation 3

while source/function scope and bounded views limit what the editor sees. In the strict CSA comparison, collection is precomputed and the visible view is frozen, so its result does not measure demand-driven routing.

Rule-guided preflight and role requests. The implementation has no separately trained or prompted patch-mechanism classifier. `PatchFactsExtractor` parses the patch and metadata for affected functions, added guards, risky calls, and type widenings. `EvidencePlanner` takes a primary pattern from analysis metadata when available or infers it from those facts, then assigns fixed priorities to requirements: patch anchoring is 100, lifetime/state requirements for double-free or use-after-free are 92/89, and a guard for null or bounds cases is 90. It sorts by descending priority with evidence type as a tie-breaker and retains at most eight planned requirements in the default configuration. Table 1 summarizes mechanism-to-role examples; it is not a classifier confusion matrix or a promise that every role is analyzer-internal. The normalizer does not learn or re-rank facts. In the general workflow the editor may request evidence types, with duplicate and request-count limits. For the audited CSA comparison, static records are precomputed and additional requests disabled. The selected v43 implementation resolves compiler references into at most four function-local CFG witness cards, selected with patch/report scope; graph dominance is not path feasibility. It contrasts compiler/checker state associations and helper spelling with the detector, appending available execution observations. No-internal receives neither native witness nor dynamic views, but keeps patch, frozen source and ordinary feedback. This bounded deterministic exposure is not a classifier or a measured optimal routing policy.

4.2 Evidence Sources and Roles

Four collectors execute in sequence; Figure 2b displays their outputs and Figure 2c records provenance. **Patch preflight** locates changed files/hunks and mechanism context. **Source extraction** resolves function, guard, call, state and build context in bounded views, without repository-scale dumps.

Analyzer evidence is classified by its actual origin, not by the collector’s name. We call a record *analyzer-internal* only if its payload is derived from a backend-produced intermediate representation, identifies the interface and raw output, and is bound to a source revision, file, and function. In the measured CSA cohort the interface is Clang 18’s `debug.DumpCFG` and `debug.DumpCallGraph`;

Table 1. Rule-guided evidence requirements by patch mechanism (representative); names are persisted record types, not gold labels or measured classifier outputs.

Patch mechanism	Possible record types	Backend
Cleanup edge / double free	allocation_lifecycle, state_transition, path_guard	CSA
Bounds widening / overflow	semantic_slice, dataflow_candidate, state_transition	CSA/CodeQL
Initialization / null deref	state_transition, path_guard	CSA
Use-after-free / escape	allocation_lifecycle, call_chain	CSA/CodeQL
API misuse / missing check	semantic_slice, call_chain	CodeQL

the persisted UTF-8 dump is parsed into schema `semweaver.csa_cfg_snapshot.v1`, with a raw-output SHA-256 and source-scope binding. These dumps provide CFG branches and value bindings and call edges. They do *not* themselves prove a path-feasible lifetime or ownership transition. Analyzer diagnostics and alert counts are *analyzer output*, not internal state. Patch diffs, source windows, and compile-database-assisted source extraction are *source-derived* fallbacks even when a CSA or CodeQL collector emits them. A live CodeQL database query could meet the internal-evidence definition if its query, database revision, and tuples were retained; we do not attribute source-window-derived CodeQL facts to that category.

The selected implementation also observes instrumented CSA callbacks, abstract values/regions and checker-state associations. Raw JSONL uses `semweaver.checker_trace.v2`; bounded feedback uses `semweaver.trace_feedback.v1`. This is not concrete execution or exhaustive symbolic-state proof. Original/instrumented scans must preserve diagnostics on both revisions; caps, live fixtures and input hashes distinguish valid absence from broken instrumentation.

Extraction must handle verbose, backend-specific intermediates and instrumentation that may shadow identifiers, exceed caps or see no eligible callback. Revision, architecture, scope and parity checks retain missing observations as unavailable. Table 2 partitions 193 records across all 39 subjects. Its 81 native records comprise 45 guards, 27 state-transition and 9 allocation-lifecycle records; all 61 source-derived records are patch diffs, not source-window fallbacks. Attached context is not itself native. Dynamic availability is per-attempt (RQ3).

Table 2. Evidence primitives and provenance in the audited 39-case CSA cohort. One record may contain several CFG primitives; counts are by record origin, not primitive.

Origin	What the persisted record contains	Records
Analyzer-internal	Clang CFG guards, value bindings and call edges; raw-output and scope binding	81
Analyzer output	CSA diagnostics and paired validation reports; no internal state	51
Source-derived	Patch-diff records here; other interfaces may emit source-window fallbacks	61

Validation feedback records compilation, execution health and paired warnings: symptoms to diagnose, not semantic ground truth.

The seven persisted types are `patch_fact` (changed statements), `semantic_slice` (bounded statements, guards, API terms and symbols), `dataflow_candidate` (possible propagation), `call_chain` (call context), `path_guard` (control), `allocation_lifecycle` (resources), and `state_`

transition (abstract state). “Barrier,” “type filter,” and “API usage” interpret record content; they are neither extra types nor proven path feasibility.

When a selected role yields no evidence, the bundle retains an explicit missing-evidence marker rather than treating the absence silently, so the refinement agent knows what was requested but unavailable.

4.3 Evidence Bundle Schema

The normalizer merges all collected facts into the EvidenceBundle shown in Figure 2c. Each fact becomes an EvidenceRecord carrying a type, the contributing analyzer, scope, source location, payload, provenance, and an optional EvidenceSlice (the compact semantic content consumed by the agent). The schema separates program facts from analysis rules in the spirit of relational analyzer frameworks [2, 5, 24], but uses a smaller role vocabulary tailored to checker refinement. Formally:

$$\begin{aligned} B &= \text{Norm}(E_p \cup E_s \cup E_a \cup E_v), \\ E_a &= E_{\text{CSA}} \cup E_{\text{CodeQL}}, \\ r \in B.\text{records} &= \langle \text{type}, \text{analyzer}, \text{scope}, \\ &\quad \text{location}, \text{payload}, \text{provenance}, \text{slice} \rangle. \end{aligned} \tag{1}$$

Here E_p contains patch facts, E_s bounded source slices, E_a analyzer-internal representations or outputs filtered by selected roles (Section 4.1), and E_v validation outcomes. Each record’s provenance has one of three origins: analyzer_internal, analyzer_output, or source_derived. The normalizer deduplicates records and marks missing evidence; origin is neither a confidence score nor a proof of semantic correctness.

The type-to-backend mapping reflects available interfaces: CSA CFG/call-graph dumps inform guard, state, and call-chain records; CodeQL queries can support flow and relation records when a live database is available. These are record types, not seven distinct backend analyses. Cross-function call edges provide context, but the present CSA dump does not establish an interprocedural path or ownership proof. Provenance remains visible when a type is populated only by a fallback.

The bundle is bounded by design: scope and selected views limit raw dumps. The matched pipeline contrast and repeats remain subject to selection history, stochastic draws and bundled native components; they do not isolate each representation’s causal effect.

4.4 Paired-Warning Refinement

The refinement loop (Figure 2d) receives the current detector, patch, baseline validation notes, and a bounded role-specific evidence view. The interface supports requesting further roles from the persisted bundle, but the provenance-audited matched condition freezes the visible view before the edit and disables additional agent requests. Thus its measured effect is a contrast between preselected native-evidence and no-internal views, not a test of dynamic evidence retrieval. This keeps the prompt bounded and the two conditions auditable.

The agent’s tool-mediated loop draws on recent LLM agent and self-refinement architectures but constrains actions to attached evidence, detector edits, and validation commands [40, 49, 52, 59]. Where evidence requests are enabled, they replay attached records rather than launch open-ended repository exploration; the request count is capped and unused or missing evidence is logged. In the matched condition, request count is zero by protocol.

Model/programmatic boundary. LLMs may produce patch-analysis metadata, checker edits and enabled role requests. Preflight rules, raw-output hashing/scope binding, normalization, compilation, paired replay and acceptance are programmatic. The LLM neither produces backend

dumps nor assigns origin or PDS. The audit retains exact starting checkers, prompts/replies, visible roles, raw evidence, candidates and decisions; model explanations do not certify analyzer origin. Editor prompts provide current/starting checker and patch paths, bounded source/evidence, baseline V/F symptoms, backend identity and edit/validation rules. They require in-place mechanism-based edits rather than patch-spelling matches, preservation of vulnerable triggers, and compilation before paired acceptance. Repair prompts supply actual compiler diagnostics. The artifact indexes available English templates and rendered exchanges; missing historical raw exchanges are marked, never reconstructed.

Retention gate. Changed candidates must compile and receive normal paired execution. Equation 3 retains warning recovery from silence, or lower fixed reports with a nonzero vulnerable signal. PDS is stronger but not mandatory. The native-only controlled-fixture guard preserves required existing hits; it is not a target oracle. Silencing an existing vulnerable signal is rejected. A comparative disadvantage against control may instead reflect failure to recover from initial silence or a smaller improvement.

Feedback loop. The next reply edits the latest attempt using build/paired feedback; the best retained detector is separate. Rejected/invalid edits remain visible. Algorithm 1 and the evaluation protocol specify stopping. Interruptions remain unscored and are repaired without resampling consumed prefixes. The noisy-15 view reuses full-cohort outputs, not a separate per-case-budget experiment.

Backend extension surface. An adapter must provide registration, evidence/provenance mapping, dependencies, execution/result decoding, and paired validation. Shared modules still contain backend branches. Four enumerated CodeQL-specific modules occupy 2,438 physical lines, including comments and generation code: an existing footprint, not code to rewrite for each backend. With a usable analyzer and patch pair, we estimate that a familiar developer using Codex can complete basic adaptation within an afternoon by reusing these interfaces. This is a planning estimate, not measured labor or a scalability result. New native-evidence semantics and broad validation require separate work. The artifact lists prerequisites and tasks. A new bug mechanism needs a role mapping and safe/unsafe fixtures, and a new extractor only for unavailable relations.

5 Evaluation

We evaluate automated patch-related warning recovery and report reduction, not whole-kernel bug discovery. Checker outcomes and execution health are recorded separately.

5.1 Research Questions

RQ1: Generated CSA checkers. From identical starting artifacts, how do KNighter’s actual loop and SEMWEAVER affect vulnerable-version warning coverage, fixed-side report burden and paired discrimination?

RQ2: Cross-backend feasibility and cost. Can an archived automatic CodeQL query be refined and independently replayed, and what integration/execution costs are observable?

RQ3: Evidence sensitivity. How do native/no-internal outcomes differ across all 39 subjects and three paired decodes of the original 12?

RQ4: Model configuration sensitivity. How do three editor configurations perform on those 12 subjects under common reply/output ceilings?

5.2 Subjects

E1 freezes one generated CSA checker per selected Linux commit from KNighter’s artifacts. Of 286 records on 60 commits, 241 on 39 commits lack an upstream refinement-status row. Per commit, the fixed input rule ranks by upstream TP, then TN, then lowest checker index, taking nonempty

repaired source. This non-random ranking selects inputs; independent paired replay supplies outcomes. Failed/noisy refinements do not remove subjects. Initially 15 checkers warn on both revisions and 24 are silent on both; zero PDS is the starting state, not KNighter’s refinement result. No manual completion enters scored portfolios.

All 39 exercise actual KNighter refinement and native/no-internal SEMWEAVER. The noisy 15 are the complete preexisting report-refinement stratum, not a result-selected subset. For the other 24, KNighter’s no-report branch compiles without model edits, whereas SEMWEAVER attempts recovery. This system comparison does not imply KNighter attempted and failed 24 recovery edits.

The selected whole implementation is v43, chosen after comparing two configurations on six experimental subjects. Exact compatible executions on those subjects are reused in E1; selection probes are not an independent confirmation set. Source revisions, reuse equivalence and earlier trials remain available. G04 uses the architecture-correct ARM64 scope and evidence, not a zero substituted for earlier wrong-object execution.

Figure 3 summarizes E1–E4. E2 uses an automatic CodeQL input from an archived independent generator; historical mixed/manual 20-row claims are withdrawn. CodeQL is neither pooled with CSA nor compared against KNighter. E3 covers all 39 main pairs plus the repeated subset; E4 is native-only, one decode per model.

The 12-subject selection uses starting status, not refinement outcomes. Within the original 14 warning-positive/fixed-noisy and 25 vulnerable-silent cases, we sort SHA-256 hashes of a fixed, artifact-recorded salt plus case ID and take the first 4 and 8, respectively. Later scan repair corrected G08 from 0/0 to 2/2: the unchanged subset becomes 5 noisy and 7 silent, with no replacement. Membership predates this final round; selection is not stratified by vulnerability class and includes no buffer-overflow/out-of-bound cases. Repeated decodes are not independent bugs; E4 is neither full-39 nor multi-run cross-model robustness.

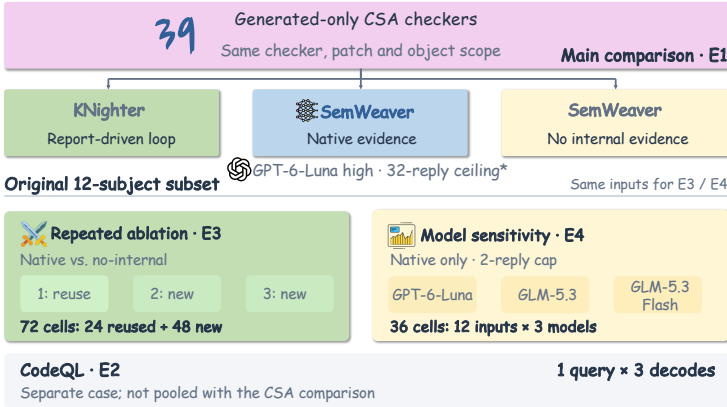


Fig. 3. **Experimental design and reuse.** E3’s first decode reuses E1; E4 uses the same 12 inputs, not another cohort. Cells are not independent bugs. *Audited natural-stop reuses retain original caps; shared ceilings do not imply equal realized effort.

5.3 Metrics

Primary outcomes are vulnerable-version warning presence and fixed-side report burden. Let $n_v(D)$ and $n_f(D)$ count warnings on the same vulnerable/fixed patch-related object scope. The

stronger secondary outcome PDS is:

$$\begin{aligned} H_v(D, P, V) &= \#[\text{alerts}(D, V_{\text{vuln}}, P) > 0], \\ S_f(D, P, V) &= \#[\text{alerts}(D, V_{\text{fix}}, P) = 0], \\ \text{PDS}(D, P, V) &= H_v(D, P, V) \wedge S_f(D, P, V). \end{aligned} \quad (2)$$

PDS is not required for a useful edit. *Warning recovery* makes an initially silent vulnerable revision warning-positive, possibly with fixed noise; *report reduction* lowers fixed reports while preserving vulnerable warning presence. Silencing a previously positive vulnerable revision is rejected. *Comparative disadvantage* means another arm retains a better result, not necessarily regression from the starting checker.

For retained detector D^* and normally paired-executed candidate C , the improvement rule is:

$$\begin{aligned} \text{Accept}(C, D^*) &= \text{AnalyzerOK}(C) \wedge H_v(C) \\ &\wedge (\neg H_v(D^*) \vee n_f(C) < n_f(D^*)) \wedge Q(C). \end{aligned} \quad (3)$$

The editor’s local gate requires a changed artifact before paired assessment. Q preserves required hits on controlled array fixtures in native; it is identically true for no-internal. This native-only guard records zero rejections in E1/E3, but its overhead and policy asymmetry remain disclosed. It is not a target-location oracle.

We binarize each checker/revision by warning presence. Each vulnerable revision is one positive and its fixed revision one negative; precision, recall and $F_1 = 2TP/(2TP + FP + FN)$ describe 78 patch-version decisions, not individual bug sites. Fixed reports measure inspection burden, not confirmed false-positive bugs. G08 warns on allocation failure rather than the registration-failure leak repaired by its patch; G24’s remaining warnings have unverified target alignment. No exhaustive target-recall claim follows. Failed scans are unscored, never substituted as zero warnings.

5.4 Experimental Environment

CSA uses pinned Clang/LLVM 18.1.3 and Python 3.12.3; CodeQL uses 2.26.4, codeql/cpp-all 8.0.3 and no-build databases. Each Linux subject uses its patch commit and parent with allyesconfig, architecture-correct objects and identical starting/baseline/candidate scope. Manifests bind code, evidence and raw paired outputs. Requested model labels and high effort are retained; service labels do not establish immutable revisions. Configured temperature 0 is not enforced sampling control: both Responses adapters omit it. We claim no fixed seed or deterministic decoding.

Generative AI in research. LLMs generated starting checkers and edit candidates. Extraction, provenance checks, compilation, paired scans and numerical outcomes are programmatic. Codex assisted with drivers, audit/analysis code and perturbation preparation; it supplies no ground-truth target labels. Original messages, English reading companions, candidates and script hashes are retained; authors verify AI-assisted code and interpretations.

5.5 Baselines and Variants

E1 native, no-internal and KNighter share starting checkers, requested GPT-6-Luna high and a 32-reply ceiling. KNighter retains its five-report/two-outer-attempt/report-driven algorithm; its actual emitted artifacts receive independent scans. Its 24 no-report branches consume no reply. Two natural one-response No-FP stops are equivalence-audited reuses with their original eight-reply ceiling retained, not relabelled as fresh32-reply runs; termination occurs before either ceiling. The 15-case view uses these same outputs, not a favorable per-case allocation. Equal maximum replies do not mean equal actual calls, tokens or CPU work.

Within SEMWEAVER, no-internal withholds native compiler and checker-execution views while keeping checker, patch, frozen source context, build/paired feedback and stopping rule. Native receives bounded CFG witness/mismatch views and available dynamics; absence is explicit. E3 repeats the original 12 at 32 replies and 16,384 output tokens per call. E4 evaluates GPT-6-Luna high, GLM-5.3 high and GLM-5.3-Flash high with 2 replies and 65,536 output tokens per call. Actual GLM effort fields are retained. E4 is limited-budget configuration sensitivity, not equal-compute model ranking or cross-model native/control causality. Earlier default-effort, smaller-output and outside-subset histories stay separate.

5.6 Protocol

Each permitted reply is an edit attempt; compilable changed candidates receive independent paired replay. The next call starts from the latest attempted artifact and its failure/paired feedback, while the best retained detector is separate. Normal candidates follow Equation 3. Stopping occurs at positive/fixed-silent saturation, three valid validations without improvement, three consecutive invalid model attempts, or the reply cap. Rejected edits, compile/format failures, observer unavailability, provider interruptions and invalid scans remain distinct.

Repairs preserve original failures and consumed prefixes. Thinking-only received replies are charged, not resent free. One GLM reply contained an explicit action and three edits but omitted explanatory summary; network-disabled replay supplied only an empty summary, reproduced exact model-specified code and made zero new calls. Healthy zero-callback probes use static evidence without invented traces. Repairs do not authorize manual scored-code edits, changed budgets or failed scans counted as negatives.

6 Results

6.1 RQ1: Generated CSA Checkers

Table 3 reports all 39 subjects and the preexisting 15-case noisy stratum. Zero starting PDS is an entry condition, not KNighter’s refinement result. Every portfolio retains the 15 initially warning-positive vulnerable revisions. Among the 24 initially silent subjects, native recovers 8, no-internal 6, and KNighter 0 under its actual no-report policy. These are warning signals, not verified target-recall counts.

Table 3. CSA comparison: (a) all 39 subjects; (b) the complete preexisting 15-case fixed-noisy stratum, reusing the same outputs. TP/FP/FN are version-warning decisions; fixed reports measure burden and replies count received responses. Color shades 1 – F_1 at .25/.50/.75, not significance.

(a) Full cohort: 39 subjects

Condition	Version outcomes				F_1	Burden / effort	
	PDS	TP	FP	FN		Fixed reports	Replies
Unmodified	0	15	15	24	.435	73	0
KNighter loop	2	15	13	24	.448	69	179
SEMWEAVER, native	5	23	18	16	.5750	123	150
SEMWEAVER, no internal	8	21	13	18	.5753	68	133

(b) Noisy 15: all methods retain TP=15, PDS=2 and $F_1 = .6977$

	KNighter loop	Native	No internal
Fixed reports	69	44	68

Warning coverage and report burden move in different directions: native’s 123 fixed reports exceed no-internal’s 68 and KNighter’s 69; its precision is 0.561 versus 0.618 for no-internal. The

nearly tied F_1 summarizes different coverage/noise portfolios and does not establish native superiority. G12 contributes 73 distinct fixed-side warning locations, not duplicates; its broader matching yields 74 vulnerable and 73 fixed reports. This unfavorable tradeoff remains in the aggregate.

In the complete 15-case fixed-noisy stratum (Table 3b), native reduces reports by 25 (36.2%) against KNighter and 24 (35.3%) against control. The effect is concentrated in G24’s $27 \rightarrow 2$ fixed reports; other small differences offset one another. Identical binary fixed positivity and F_1 thus coexist with different report burdens. G24’s target preservation is unverified, so lower burden does not guarantee sound suppression.

6.2 RQ2: Cross-Backend Feasibility

A separate archived automatic ImageMagick CodeQL query starts at 0/0 vulnerable/fixed rows. Table 4 includes two 8/0 recoveries and the third decode’s 22/22 outcome. The successful edit corrects relative-path matching while keeping exact file/function/local-variable filters. This demonstrates patch-local operation on a second format, not broad backend transfer or native-evidence causality. A second query already starts at 2/0 and is not counted as a gain. KNighter is not a CodeQL baseline.

Table 4. Three automatic CodeQL decodes of one query. Counts are vulnerable/fixed output rows; time is recorded agent-round duration, excluding separate paired replay.

Decode	V/F rows	Replies	Tokens	Agent time (s)
1	8/0	1	15,122	87.87
2	8/0	1	14,345	42.74
3	22/22	8	110,282	516.77

Second-backend costs. Table 4 reports the original model runs. Three warm paired replays per query preserve 0/0 and 8/0. Median query-plus-decode time is 12.57 s initially and 36.51 s after refinement with two threads; peak process RSS is 976/1,078 MiB. Database-creation log intervals span 31/30 s, excluding unlogged startup. Developer labor is unmeasured. These one-case existing-environment observations are neither a speedup, cold-install benchmark nor whole-project scalability estimate.

6.3 RQ3: Internal-Evidence Sensitivity

Table 2 reports initial record provenance; Section 4.2 defines native types and source-derived context. Dynamic availability has a separate denominator: 103 of 150 unique native-chain attempts have usable hash-bound observations; 11 are unavailable due to candidate compilation, 18 unsupported context, 13 zero scoped callbacks, 1 reproduced candidate crash and 4 capped optional observations. None establishes safety from absence.

Across 39 subjects, native has six comparative benefits, five disadvantages and 28 ties/other outcomes. The associated edits expose concrete differences in checker behavior: cleanup-wrapper recognition in G19, nested dereference/binding recovery in G25, and null/region/frame handling in G24. G19 adds `__free_kfree` to a cleanup predicate, yielding 15/0 versus control’s 0/0; 15 reports are not 15 bugs. Conversely, control recovers symbolic-region handling in G23 and persistent-field release recognition in G38 without native records. G12 drops CPU-role restrictions, explaining its broad matching and noise. These code observations distinguish repairs from broader matching; a record’s presence or an LLM explanation alone does not prove edit causality.

Figure 4 shows five subjects tying in all three decodes and seven with mixed directions; none benefits comparatively in all three. G19’s native 15/0 recurs, but control catches up in its third

Subject	Decode 1 (E1)			Decode 2			Decode 3		
G04	=	0/0	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G08	=	1/1	: 1/1	—	2/2	: 1/1	=	1/1	: 1/1
G09	+	1/1	: 0/0	=	1/1	: 1/1	=	1/1	: 1/1
G13	=	0/0	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G14	—	2/2	: 2/1	=	2/1	: 2/1	=	2/1	: 2/1
G15	=	9/9	: 9/9	=	9/9	: 9/9	=	9/9	: 9/9
G19	+	15/0	: 0/0	+	15/0	: 0/0	=	15/0	: 15/0
G24	+	2/2	: 27/27	=	3/3	: 3/3	=	27/27	: 27/27
G25	+	2/2	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G36	=	1/1	: 1/1	=	1/1	: 1/1	=	1/1	: 1/1
G37	=	0/0	: 0/0	=	0/0	: 0/0	=	0/0	: 0/0
G38	—	0/0	: 1/0	=	1/0	: 1/0	=	1/0	: 1/0

Native V/F : no-internal V/F. + benefit; — disadvantage; = tie/other.

Recovery: native 4 / control 1; report reduction: native 1 / control 2; ties/other 28.

Fig. 4. E3: all 36 paired decodes on 12 subjects; cells compare native and no-internal V/F counts. Decode 1 reuses E1; the remaining decodes add 48 condition cells. Color and sign encode the comparative direction.

decode. G24’s main reduction does not repeat. G14’s $3/3 \rightarrow 2/2$ improves its own start but trails control’s $2/1$; later decodes tie. In the main comparison, G05/G23/G38 retain initially 0/0 in native while control recovers warnings, so their disadvantage is not loss of an initial signal. Repeated subjects, selection/reuse and native-only gating limit stronger attribution. Realized call differences under a shared stopping rule are outcomes/mediators, not independently assigned confounds.

6.4 RQ4: Model Configuration Sensitivity

Table 5 uses the original 12 subjects, native only, one decode per model, 2-reply and 65,536-output ceilings with requested high effort. All configurations retain the five initial vulnerable warning signals and recover additional versions. Flash has the strongest observed outcome here, but one decode on this subset does not establish general ranking or variance-controlled model robustness.

Table 5. Limited-budget native sensitivity: 12 subjects per model, not full-39 robustness. Computation differs despite common reply/output ceilings. F_1 shading follows Table 3.

Editor	TP	FP	PDS	F_1	Fixed reports	Replies
Unmodified	5	5	0	.455	42	0
GPT-6-Luna high	7	6	1	.560	40	24
GLM-5.3 high	7	5	2	.583	16	23
GLM-5.3-Flash high	9	5	4	.692	11	21

Received replies total 24/23/21 respectively. Capped, format-invalid and compile-failed replies remain charged; a healthy previous detector is retained when no valid improvement occurs. These are complete workflow outcomes, not successful code generation on every call. No no-internal arm is run across models, so E4 cannot isolate cross-model evidence effects.

6.5 Qualitative Findings

The automatic motivating checker retains AST traversal and adds actual-capacity reasoning, with 14 received replies and fixture-guided corrections. Its 114 finite successes are separate from the

main one-response checker’s 74/114. The case ledger retains all 11 non-tied main comparisons, selected diffs and intermediate outcomes. Three positive selected candidates have usable dynamics (G19/G24/G25), and three use static records with unavailable dynamics (G09/G12/G35). All six gains therefore cannot be credited to dynamic traces. The ledger makes both useful completion and costly broadening inspectable, connecting aggregate outcomes to concrete edits. It supports conditional observations; universal precision improvement, target preservation and unseen-family recall remain unproved.

7 Discussion

Why post-synthesis and LLM-generated checkers? Holding the checker fixed attributes improvement to an edit rather than a new generation draw. The interface can also accept a compatible handwritten checker and paired target, although that population and evidence-assisted fresh generation remain unevaluated.

Evidence as context compression. Provenance-tagged, patch-scoped views bound the editor’s context instead of exposing full repository and analyzer dumps. This frames refinement as specification completion rather than report suppression [12, 39, 41, 42, 53]; the present study does not isolate compression efficacy.

Backend complementarity. The CSA study and separate E2 CodeQL replay exercise two detector formats. They do not show that every evidence role was populated from an analyzer-internal representation. In the audited CSA cohort, the verified native interface is a CFG/call-graph dump; source-derived and diagnostic records remain separately labelled. A CodeQL-native coverage claim requires retained database-query provenance.

Interpreting benefits and costs. G19 exposes a missing cleanup-wrapper case, while G12 recovers warnings through broader matching with substantial fixed noise. G24 reduces report counts with target preservation unresolved. These observed edits suggest inspecting the changed predicate alongside both warning counts: recovery, lower burden and stronger discrimination can diverge. They do not establish a rule predicting when native evidence will help. Over-pruning is rejected; the retention rule permits noisy recovery. Missing observations, unsupported interfaces, invalid output and compile errors remain explicit, and failed execution never becomes a fabricated zero-warning result.

Limited semantic generalization. The motivating checker passes finite, fixture-guided perturbations, not unseen-family recall tests. Section 3 details its extent/index reasoning, tested transformations, separate budget and the weaker main checker. Arbitrary aliasing and dynamic extents remain outside scope.

Implications for evaluation and use. The paired comparison makes recovery and inspection burden jointly observable. RQ1’s noisy-stratum reduction is concentrated in G24; native’s full-cohort F_1 nearly ties control despite higher coverage and noise. E3 shows why a favorable edit and a repeatable comparative advantage require separate evidence. Static views cover all 39; dynamics vary by attempt. Changed forks or versions need stability and target-location checks. Each tested model configuration improves coverage and report burden on 12 subjects, without cross-model repetitions. Warm CodeQL costs, code footprint and estimated afternoon adaptation establish neither measured labor nor whole-kernel scalability.

8 Threats to Validity

Internal. Collection, editing or validation errors can create spurious outcomes. Candidate compile/format failures are distinct from invalid scans; scanner crashes remain unscored with replies charged and are repaired without resetting prefixes. Zero-callback/capped observations remain unavailable, not safe. Raw hashes, scope/parity checks and independent paired replay reduce but do

not eliminate implementation risks. All scored checker edits are automatic; old manual/mixed rows are excluded. The native-only fixture guard rejects zero E1/E3 candidates, but is not an identical-policy or target-correctness guarantee. Selection probes overlap the evaluated cohort; reused executions remain labelled. Repeats do not create fresh-subject confirmation or isolate static/dynamic components. Configured temperature is not proof of transmitted sampling control.

Construct. Coverage, report burden and secondary PDS measure paired-version warnings, not independently adjudicated target findings; G08 is a counterexample and G24 leaves an ambiguity about suppression and target preservation. F_1 uses 78 version decisions, not deployment precision or target recall. A recovered warning may be noisy, notably G12. Scope includes patch-related objects rather than only the changed line, but does not certify semantics on other sites. The partial gate preserves nonzero vulnerable warning presence, not the entire warning set or every target location.

External. The 39-case one-per-commit cohort is not a random screen of all 286 upstream records or independent projects. TP/TN-based input selection may favor some artifacts; no population effect rate is claimed. The complete 15-case noisy stratum and 24 silent subjects have different tasks; KNighter does not attempt recovery without reports. E4 covers only the original 12-subject subset, omitting bounds categories, with one decode per configuration. Requested aliases do not identify immutable revisions. The CodeQL example is one patch-local case with inherited filters, not broad backend transfer. Perturbations informed the motivating edits and are not an unseen-code or future-version evaluation.

Conclusion. Repeated decodes are grouped by subject, not treated as independent bugs. Small non-random samples, prior configuration selection and stochastic outcomes limit uncertainty estimation. Native/control F_1 is nearly equal and no E3 subject benefits comparatively in all three decodes. We report all weaker/noisy outcomes and claim conditional observations, not statistically significant universal superiority.

9 Related Work

Static Analysis and Checker Languages. Path-sensitive analyzers, abstract interpreters and symbolic engines provide guards, states, paths and diagnostics [3, 6, 13, 15], with industrial deployments [4, 7, 48]. Relational frameworks query program facts through rules and points-to specifications [2, 5, 24]; semantic patching encodes recurring systems-code transformations [43]. SEMWEAVER introduces no analyzer or query language: it edits LLM-generated checkers using analyzer-native facts for semantic completion.

Vulnerability Detection and Patch Evidence. Static vulnerability detection has long used taint and data-flow reasoning [1, 25, 36], graph and AST representations [55, 56], and learning-based token-, slice-, and line-level models [17, 34, 35, 46], though datasets and evaluations show that semantic robustness remains difficult [8, 16]. Clone- and patch-driven systems use historical fixes as compact witnesses [22, 28, 33]. These motivate our use of patches as specifications, but SEMWEAVER targets a different artifact: an already generated checker whose fixed-side behavior reveals missing semantics.

LLM-Based Code and Security Reasoning. Code LLMs synthesize programs [11], and agents such as RepoAudit audit repositories [19]. LLMDFA decomposes compilation-free dataflow analysis and uses synthesized tools for extraction and feasibility checks [54]. Generated artifacts still require security validation [26, 44, 45]. SEMWEAVER edits a static-analysis artifact; paired analyzer executions, not the model’s explanation, determine acceptance.

Checker Synthesis and Feedback-Guided Repair. Patch-driven checker synthesis, especially KNighter [57], is the closest prior work. LLMs transform historical security fixes into specialized analyzers that compile, execute, and are reused. SEMWEAVER is complementary because it assumes

such a detector exists and refines it post-synthesis. A generator’s validation feedback reveals which side a detector fires on but not whether the missing predicate is a guard, barrier, flow, state, lifetime, type filter, or API-usage relation.

The loop also relates to repair, synthesis and counterexample-guided workflows using compiler/test/verifier feedback [12, 27, 30, 38, 39, 41, 42, 53], and iterative LLM feedback, tool use and search [40, 49, 52, 58, 59]. Here the edited artifact is a detector, not the vulnerable program. Paired execution retains warning recovery or report reduction with vulnerable warning presence, not compilation or suppression alone; this does not prove target detection.

Analyzer Context for LLMs and Alert Triage. LLift combines static analysis and LLMs for post-constraint-guided use-before-initialization reasoning [31]. Chapman et al. interleave intermediate results and prompts to infer error specifications [9]; IRIS infers taint specifications and analyzes CodeQL context [32]. LLM4PFA filters reports through interprocedural feasibility reasoning [14], and ZeroFalse enriches alerts before LLM adjudication [21]. VulGenie combines patch-dependency and attack/defense value-flow analysis with LLM-assisted API-rule synthesis and adaptive Java detection [10]. We claim neither first use of analyzer information nor new path-feasibility reasoning. Our operation completes an existing checker using auditable analyzer and checker context with paired warning outcomes, not a universal causal or precision claim. Evidence origin, selected edits, failed attempts and scope remain inspectable.

10 Conclusion

SEMWEAVER combines an auditable post-synthesis checker editor with an empirical account of the benefits and costs of analyzer context. Provenance-labelled compiler/checker evidence and paired validation make the resulting edits and outcomes inspectable. Across 39 generated-only CSA subjects, native GPT reaches 23 warning-positive vulnerable revisions and version-level $F_1 = 0.575$, versus KNighter’s 15 and 0.448. In the complete 15-case noisy stratum, fixed reports fall from KNighter’s 69 to 44 with all vulnerable warning signals retained, primarily through one subject. Overall native noise increases, F_1 nearly ties no-internal, and repeated benefits depend on sampling. A separate automatic CodeQL example and three-model subset provide bounded feasibility and sensitivity evidence. The study characterizes conditional checker-completion tradeoffs across warning recovery and report burden; universal precision gains, target-recall proof and unseen-site generalization remain unestablished.

Acknowledgments

In paper writing, the authors used generative AI tools (ChatGPT and Claude) solely for language polishing and clarity-oriented editing.

Data Availability

The anonymized repository is at <https://anonymous.4open.science/r/SemWeaver-FD1E/>. The English package contains implementation, frozen inputs, raw-output/provenance bindings, both systems’ outputs, original exchanges with English reading companions, all reported repeated/model-subset results, and offline byte/reaggregation checks. Historical manual/mixed-provenance materials are unscored. Full reruns require public Linux revisions and the pinned toolchain; the package is intended to remain public upon acceptance.

References

- [1] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 259–269. doi:10.1145/2594291.2594299
- [2] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. 2016. QL: Object-Oriented Queries on Relational Data. In *Proceedings of the 30th European Conference on Object-Oriented Programming (ECOOP)*. 2:1–2:25. doi:10.4230/LIPIcs.ECOOP.2016.2
- [3] Thomas Ball and Sriram K. Rajamani. 2002. The SLAM Project: Debugging System Software via Static Analysis. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 1–3. doi:10.1145/503272.503274
- [4] Al Bessey, Ken Block, Ben Chelf, Andy Chou, Bryan Fulton, Seth Hallem, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson Engler. 2010. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Commun. ACM* 53, 2 (2010), 66–75. doi:10.1145/1646353.1646374
- [5] Martin Bravenboer and Yannis Smaragdakis. 2009. Strictly Declarative Specification of Sophisticated Points-to Analyses. In *Proceedings of the 24th ACM SIGPLAN Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*. 243–262. doi:10.1145/1640089.1640108
- [6] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 209–224. https://www.usenix.org/legacy/event/osdi08/tech/full_papers/cadar/cadar.pdf
- [7] Cristiano Calcagno, Dino Distefano, Jérémy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O’Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. 2015. Moving Fast with Software Verification. In *Proceedings of the 7th NASA Formal Methods Symposium (NFM)*. 3–11. doi:10.1007/978-3-319-17524-9_1
- [8] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* 48, 9 (2022), 3280–3296. doi:10.1109/TSE.2021.3087402
- [9] Patrick J. Chapman, Cindy Rubio-González, and Aditya V. Thakur. 2024. Interleaving Static Analysis and LLM Prompting. In *Proceedings of the 13th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*. 9–17. doi:10.1145/3652588.3663317
- [10] Bofei Chen, Shuang Liao, Lei Zhang, Chibin Zhang, Mathias Payer, and Yuan Zhang. 2026. Patch-Guided Vulnerability Detection: Extracting Java API Security Rules via Attack–Defense Cross-Analysis. In *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, Baltimore, MD, 1847–1866. <https://www.usenix.org/conference/usenixsecurity26/presentation/chen-bofei>
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374
- [12] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *Proceedings of the 12th International Conference on Computer Aided Verification (CAV)*. 154–169. doi:10.1007/10722167_15
- [13] Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. 2005. The ASTRÉE Analyzer. In *Proceedings of the 14th European Symposium on Programming (ESOP)*. 21–30. doi:10.1007/978-3-540-31987-0_3
- [14] Xueying Du, Kai Yu, Chong Wang, Yi Zou, Wentai Deng, Zuoyu Ou, Xin Peng, Lingming Zhang, and Yiling Lou. 2025. Minimizing False Positives in Static Bug Detection via LLM-Enhanced Path Feasibility Analysis. *arXiv:2506.10322* <https://arxiv.org/abs/2506.10322>
- [15] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. 2001. Bugs as Deviant Behavior: A General Approach to Inferring Errors in Systems Code. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*. 57–72. doi:10.1145/502034.502041
- [16] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. 508–512. doi:10.1145/3379597.3387501
- [17] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-Based Line-Level Vulnerability Prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. 608–620. doi:10.1145/3524842.3528452
- [18] GitHub. 2026. CodeQL Documentation. <https://codeql.github.com/docs/>. Accessed: Jun. 27, 2026.

- [19] Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, and Xiangyu Zhang. 2025. RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 21083–21100. <https://proceedings.mlr.press/v267/guo25n.html>
- [20] Sarah Heckman and Laurie Williams. 2011. A Systematic Literature Review of Actionable Alert Identification Techniques for Automated Static Code Analysis. *Information and Software Technology* 53, 4 (2011), 363–387. doi:10.1016/j.infsof.2010.12.007
- [21] Mohsen Iranmanesh, Sina Moradi Sabet, Sina Marefat, Ali Javidi Ghasr, Allison Wilson, Iman Sharafaldin, and Mohammad A. Tayebi. 2025. ZeroFalse: Improving Precision in Static Analysis with LLMs. arXiv:2510.02534 <https://arxiv.org/abs/2510.02534>
- [22] Jiyong Jang, Abeer Agrawal, and David Brumley. 2012. ReDeBug: Finding Unpatched Code Clones in Entire OS Distributions. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy*. 48–62. doi:10.1109/SP.2012.13
- [23] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why Don't Software Developers Use Static Analysis Tools to Find Bugs?. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, San Francisco, CA, 672–681. doi:10.1109/ICSE.2013.6606613
- [24] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On Synthesis of Program Analyzers. In *Proceedings of the 28th International Conference on Computer Aided Verification (CAV)*. 422–430. doi:10.1007/978-3-319-41540-6_23
- [25] Nenad Jovanovic, Christopher Kruegel, and Engin Kirda. 2006. Pixy: A Static Analysis Tool for Detecting Web Application Vulnerabilities. In *Proceedings of the 2006 IEEE Symposium on Security and Privacy*. 258–263. doi:10.1109/SP.2006.29
- [26] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure Is Code Generated by ChatGPT?. In *Proceedings of the 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2445–2451. doi:10.1109/SMC53992.2023.10394237
- [27] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 802–811. doi:10.1109/ICSE.2013.6606626
- [28] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy*. 595–614. doi:10.1109/SP.2017.62
- [29] Ted Kremenek and Dawson Engler. 2003. Z-Ranking: Using Statistical Analysis to Counter the Impact of Static Analysis Approximations. In *Proceedings of the 10th International Static Analysis Symposium (SAS)*. 295–315. doi:10.1007/3-540-44898-5_16
- [30] Claire Le Goues, Michael Dewey-Vogt, Stephanie Forrest, and Westley Weimer. 2012. A Systematic Study of Automated Program Repair: Fixing 55 out of 105 Bugs for \$8 Each. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. 3–13. doi:10.1109/ICSE.2012.6227211
- [31] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1, Article 111 (2024), 26 pages. doi:10.1145/3649828
- [32] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/582d4e27fa24168f3af1f4582655034b-Abstract-Conference.html
- [33] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Hanchao Qi, and Jie Hu. 2016. VulPecker: An Automated Vulnerability Detection System Based on Code Similarity Analysis. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*. 201–213. doi:10.1145/2991079.2991102
- [34] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2022. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2022), 2244–2258. doi:10.1109/TDSC.2021.3051525
- [35] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In *Proceedings of the 25th Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2018.23158
- [36] V. Benjamin Livshits and Monica S. Lam. 2005. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Proceedings of the 14th USENIX Security Symposium*. 271–286. https://www.usenix.org/legacy/events/sec05/tech/full_papers/livshits/livshits.pdf
- [37] LLVM Project. 2026. Clang Static Analyzer Documentation. <https://clang-analyzer.llvm.org/>. Accessed: Jun. 27, 2026.
- [38] Fan Long and Martin Rinard. 2015. Staged Program Repair with Condition Synthesis. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 166–178. doi:10.1145/2786805.2786811

- [39] Fan Long and Martin Rinard. 2016. Automatic Patch Generation by Learning Correct Code. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 298–312. doi:10.1145/2837614.2837617
- [40] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 46534–46594. doi:10.52202/075280-2019
- [41] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 691–701. doi:10.1145/2884781.2884807
- [42] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. SemFix: Program Repair via Semantic Analysis. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 772–781. doi:10.1109/ICSE.2013.6606623
- [43] Yoann Padioleau, Julia L. Lawall, René Rydhof Hansen, and Gilles Muller. 2008. Documenting and Automating Colateral Evolutions in Linux Device Drivers. In *Proceedings of the 3rd ACM SIGOPS/EuroSys European Conference on Computer Systems*. 247–260. doi:10.1145/1352592.1352618
- [44] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*. 754–768. doi:10.1109/SP46214.2022.9833571
- [45] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. doi:10.1145/3576915.3623157
- [46] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated Vulnerability Detection in Source Code Using Deep Representation Learning. In *Proceedings of the 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. 757–762. doi:10.1109/ICMLA.2018.00120
- [47] Joseph R. Ruthruff, John Penix, J. David Morgenthaler, Sebastian Elbaum, and Gregg Rothermel. 2008. Predicting Accurate and Actionable Static Analysis Warnings: An Experimental Approach. In *Proceedings of the 30th International Conference on Software Engineering (ICSE)*. 341–350. doi:10.1145/1368088.1368135
- [48] Caitlin Sadowski, Jeffrey van Gogh, Ciera Jaspan, Emma Soderberg, and Collin Winter. 2015. Tricorder: Building a Program Analysis Ecosystem. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*. 598–608. doi:10.1109/ICSE.2015.76
- [49] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 68539–68551. doi:10.52202/075280-2997
- [50] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. AddressSanitizer: A Fast Address Sanity Checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*. USENIX Association, Boston, MA, 309–318. <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany>
- [51] Srinivasan Shanmugam. 2024. drm/amd/display: Fix buffer overflow in get_host_router_total_dp_tunnel_bw(). Linux kernel commit 97cba232549b9fe7e491fb60a69cf93075015f29. <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=97cba232549b9fe7e491fb60a69cf93075015f29> Commit metadata and source checked locally, September 27, 2026.
- [52] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 8634–8652. doi:10.52202/075280-0377
- [53] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 404–415. doi:10.1145/1168857.1168907
- [54] Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, Xiaoheng Xie, and Xiangyu Zhang. 2024. LLM DFA: Analyzing Dataflow in Code with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 131545–131574. doi:10.52202/079017-4181
- [55] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy*. 590–604. doi:10.1109/SP.2014.44
- [56] Fabian Yamaguchi, Markus Lottmann, and Konrad Rieck. 2012. Generalized Vulnerability Extrapolation Using Abstract Syntax Trees. In *Proceedings of the 28th Annual Computer Security Applications Conference (ACSAC)*. 359–368. doi:10.1145/2420950.2421003

[57] Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. 2025. KNighter: Transforming Static Analysis with LLM-Synthesized Checkers. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*. 655–669. doi:10.1145/3731569.3764827

[58] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 11809–11822. doi:10.52202/075280-0517

[59] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X